Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «ОИвИС»
Тема: «Конструирование моделей на базе предобученных нейронных сетей»

Выполнил:
Студент 4 курса
Группы ИИ-25
Пальчастый Я.М
Проверил:
Андренко К.В.

Брест 2026

Цель работы:
осуществлять обучение НС, сконструированных на базе предобученных архитектур НС
Общее задание
1. Для заданной выборки и архитектуры предобученной нейронной организовать процесс обучения НС, предварительно изменив структуру слоев, в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;

2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;

3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;

4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1). Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения (например, из сети Интернет) с отображением результата классификации;

5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

Вариант 8

8	CIFAR-10	Adam	MobileNet v3

Ключевые различия

Аспект	Лаба 1	Лаба 2
Датасет	CIFAR-100 (100 классов)	CIFAR-10 (10 классов)
Модели	1 CNN с нуля	SmallConvNet + MobileNetV3-Small (transfer learning)

Оптимизатор	Adam(lr=1.0, rho=0.9, wd=1e-4)	Adam(lr=1e-3)
Эпох	20	10 (scratch) + 5 (finetune)
Воспроизводимость	нет seed	random.seed, np.random.seed, torch.manual_seed
Оценка	только accuracy	accuracy + сравнение с бенчмарками (человек, SOTA)

Код программы:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image


BATCH_SIZE = 128
EPOCHS = 20
LEARNING_RATE = 1.0
RHO = 0.9

DEVICE = torch.device(
    "cuda" if torch.cuda.is_available()
    else "cpu"
)

print("Device:", DEVICE)


CIFAR100_MEAN = (0.5071, 0.4865, 0.4409)
CIFAR100_STD  = (0.2673, 0.2564, 0.2762)


transform_train = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(CIFAR100_MEAN, CIFAR100_STD)
])
```

```
transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(CIFAR100_MEAN, CIFAR100_STD)
])


train_dataset = torchvision.datasets.CIFAR100(
    root="./data",
    train=True,
    download=True,
    transform=transform_train
)

test_dataset = torchvision.datasets.CIFAR100(
    root="./data",
    train=False,
    download=True,
    transform=transform_test
)


classes = train_dataset.classes
NUM_CLASSES = len(classes)


train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=0,
    pin_memory=True
)

test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=0,
    pin_memory=True
)

print("Train images:", len(train_dataset))
print("Test images:", len(test_dataset))
print("Number of classes:", NUM_CLASSES)



class CNN(nn.Module):
    def __init__(self, num_classes=100):
        super(CNN, self).__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
```

```
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x


model = CNN(num_classes=NUM_CLASSES).to(DEVICE)
print(model)


criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(
    model.parameters(),
    lr=LEARNING_RATE,
    rho=RHO,
    weight_decay=1e-4
)

train_losses = []
test_losses = []
train_accuracies = []
test_accuracies = []

best_accuracy = 0.0


for epoch in range(EPOCHS):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:
        images = images.to(DEVICE, non_blocking=True)
        labels = labels.to(DEVICE, non_blocking=True)

        optimizer.zero_grad()

        outputs = model(images)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    train_loss = running_loss / total
    train_accuracy = 100.0 * correct / total


    model.eval()
```

```
    test_loss_sum = 0.0
    test_correct = 0
    test_total = 0

    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(DEVICE, non_blocking=True)
            labels = labels.to(DEVICE, non_blocking=True)

            outputs = model(images)
            loss = criterion(outputs, labels)

            test_loss_sum += loss.item() * images.size(0)

            _, predicted = torch.max(outputs, 1)
            test_total += labels.size(0)
            test_correct += (predicted == labels).sum().item()

    test_loss = test_loss_sum / test_total
    test_accuracy = 100.0 * test_correct / test_total

    train_losses.append(train_loss)
    test_losses.append(test_loss)
    train_accuracies.append(train_accuracy)
    test_accuracies.append(test_accuracy)

    print(
        f"Epoch [{epoch + 1}/{EPOCHS}] "
        f"Train Loss: {train_loss:.4f} "
        f"Train Acc: {train_accuracy:.2f}% "
        f"Test Loss: {test_loss:.4f} "
        f"Test Acc: {test_accuracy:.2f}%"
    )

    if test_accuracy > best_accuracy:
        best_accuracy = test_accuracy
        torch.save(model.state_dict(), "best_cifar100.pth")

print(f"\nBest test accuracy: {best_accuracy:.2f}%")


plt.figure(figsize=(10, 5))

plt.plot(range(1, EPOCHS + 1), train_losses, label="Train Loss")
plt.plot(range(1, EPOCHS + 1), test_losses, label="Test Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Изменение функции ошибки (CIFAR-100)")
plt.legend()
plt.grid()
plt.tight_layout()

plt.savefig("loss_cifar100.png", dpi=300)
plt.show()


plt.figure(figsize=(10, 5))

plt.plot(range(1, EPOCHS + 1), train_accuracies, label="Train Accuracy")
plt.plot(range(1, EPOCHS + 1), test_accuracies, label="Test Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
```

```
plt.title("Изменение точности классификации (CIFAR-100)")
plt.legend()
plt.grid()
plt.tight_layout()

plt.savefig("accuracy_cifar100.png", dpi=300)
plt.show()


model.load_state_dict(
    torch.load("best_cifar100.pth", map_location=DEVICE)
)
model.eval()



def denormalize(image):
    mean = torch.tensor(CIFAR100_MEAN).view(3, 1, 1)
    std = torch.tensor(CIFAR100_STD).view(3, 1, 1)

    image = image.cpu()
    image = image * std + mean

    return torch.clamp(image, 0, 1)



images, labels = next(iter(test_loader))
images_gpu = images.to(DEVICE)

with torch.no_grad():
    outputs = model(images_gpu)
    probabilities = torch.softmax(outputs, dim=1)
    _, predictions = torch.max(outputs, 1)

plt.figure(figsize=(12, 8))

for i in range(12):
    image = denormalize(images[i])
    image = image.permute(1, 2, 0).numpy()

    plt.subplot(3, 4, i + 1)
    plt.imshow(image)

    predicted_class = classes[predictions[i].item()]
    real_class = classes[labels[i].item()]
    probability = probabilities[i, predictions[i]].item() * 100

    plt.title(
        f"Pred: {predicted_class}\n"
        f"Real: {real_class}\n"
        f"{probability:.1f}%"
    )
    plt.axis("off")

plt.tight_layout()
plt.savefig("predictions_cifar100.png", dpi=300)
plt.show()



def predict_image(image_path):
    transform = transforms.Compose([
        transforms.Resize((32, 32)),
        transforms.ToTensor(),
```

```
        transforms.Normalize(CIFAR100_MEAN, CIFAR100_STD)
    ])

    image = Image.open(image_path).convert("RGB")
    original_image = image.copy()

    image = transform(image)
    image = image.unsqueeze(0)
    image = image.to(DEVICE)

    with torch.no_grad():
        output = model(image)
        probabilities = torch.softmax(output, dim=1)

    predicted_class = torch.argmax(probabilities, dim=1).item()
    probability = probabilities[0, predicted_class].item()

    plt.figure(figsize=(5, 5))
    plt.imshow(original_image)
    plt.title(
        f"Предсказание: {classes[predicted_class]}\n"
        f"Вероятность: {probability * 100:.2f}%"
    )
    plt.axis("off")
    plt.tight_layout()
    plt.show()

    print()
    print("Предсказанный класс:", classes[predicted_class])
    print("Вероятность:", f"{probability * 100:.2f}%")


predict_image("sea.jpg")
```

Результат:

C:\Users\arosl\PycharmProjects\OIS2\.venv\Scripts\python.exe C:\Users\arosl\PycharmProjects\OIS2\main.py

Устройство для вычислений: cuda

Обучающих изображений: 50000 | тестовых: 10000

Размер тензора одной картинки: torch.Size([3, 32, 32])

SmallConvNet(

(features): Sequential(

(0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(1): ReLU(inplace=True)

(2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)

(3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

```
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=2,
stride=2, padding=0, dilation=1,
ceil_mode=False)
    (6): Conv2d(64, 128, kernel_size=(3,
3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): MaxPool2d(kernel_size=2,
stride=2, padding=0, dilation=1,
ceil_mode=False)
  )
  (head): Sequential(
    (0): Flatten(start_dim=1, end_dim=-
1)
    (1): Linear(in_features=2048,
out_features=256, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.4, inplace=False)
    (4): Linear(in_features=256,
out_features=10, bias=True)
  )
)
Параметров в самодельной сети:
620,362
[ 1/10]  tr_loss=1.6637  tr_acc=38.67%
|   ts_loss=1.3088  ts_acc=52.34%
[ 2/10]  tr_loss=1.3255  tr_acc=52.25%
|   ts_loss=1.1021  ts_acc=60.49%
[ 3/10]  tr_loss=1.1592  tr_acc=58.25%
|   ts_loss=0.9696  ts_acc=65.85%
[ 4/10]  tr_loss=1.0504  tr_acc=62.57%
|   ts_loss=0.8938  ts_acc=67.83%
[ 5/10]  tr_loss=0.9668  tr_acc=66.06%
|   ts_loss=0.8322  ts_acc=71.15%
[ 6/10]  tr_loss=0.9116  tr_acc=67.95%
|   ts_loss=0.7688  ts_acc=73.47%
[ 7/10]  tr_loss=0.8611  tr_acc=69.74%
|   ts_loss=0.7744  ts_acc=73.16%
[ 8/10]  tr_loss=0.8179  tr_acc=71.45%
|   ts_loss=0.7408  ts_acc=74.18%
[ 9/10]  tr_loss=0.7891  tr_acc=72.57%
|   ts_loss=0.7600  ts_acc=73.83%
```

[10/10] tr_loss=0.7615 tr_acc=73.43% | ts_loss=0.6709 ts_acc=77.49%

Датасет для finetune: 50000 / 10000

Размер входа после ресайза: torch.Size([3, 224, 224])

Downloading: "https://download.pytorch.org/models/mobilenet_v3_small-047dcff4.pth" to C:\Users\arosl/.cache\torch\hub\checkpoints\mobilenet_v3_small-047dcff4.pth

100.0%

Новый классификатор: Sequential(

(0): Linear(in_features=576, out_features=1024, bias=True)

(1): Hardswish()

(2): Dropout(p=0.2, inplace=True)

(3): Linear(in_features=1024, out_features=10, bias=True)

)

Всего весов: 1,528,106 | обучаемых (только голова): 601,098

[1/5] tr_loss=0.6256 tr_acc=78.46% | ts_loss=0.4796 ts_acc=83.27%

[2/5] tr_loss=0.4841 tr_acc=83.26% | ts_loss=0.4345 ts_acc=84.93%

[3/5] tr_loss=0.4357 tr_acc=84.61% | ts_loss=0.4167 ts_acc=85.19%

[4/5] tr_loss=0.4005 tr_acc=85.92% | ts_loss=0.4188 ts_acc=85.50%

[5/5] tr_loss=0.3726 tr_acc=86.82% | ts_loss=0.3972 ts_acc=86.31%

Итог MobileNetV3 на тесте: 86.31%

Сводка

SmallConvNet : 77.49% (620,362 параметров, все обучаются)

MobileNetV3 : 86.31% (601,098 обучаемых из 1,528,106)

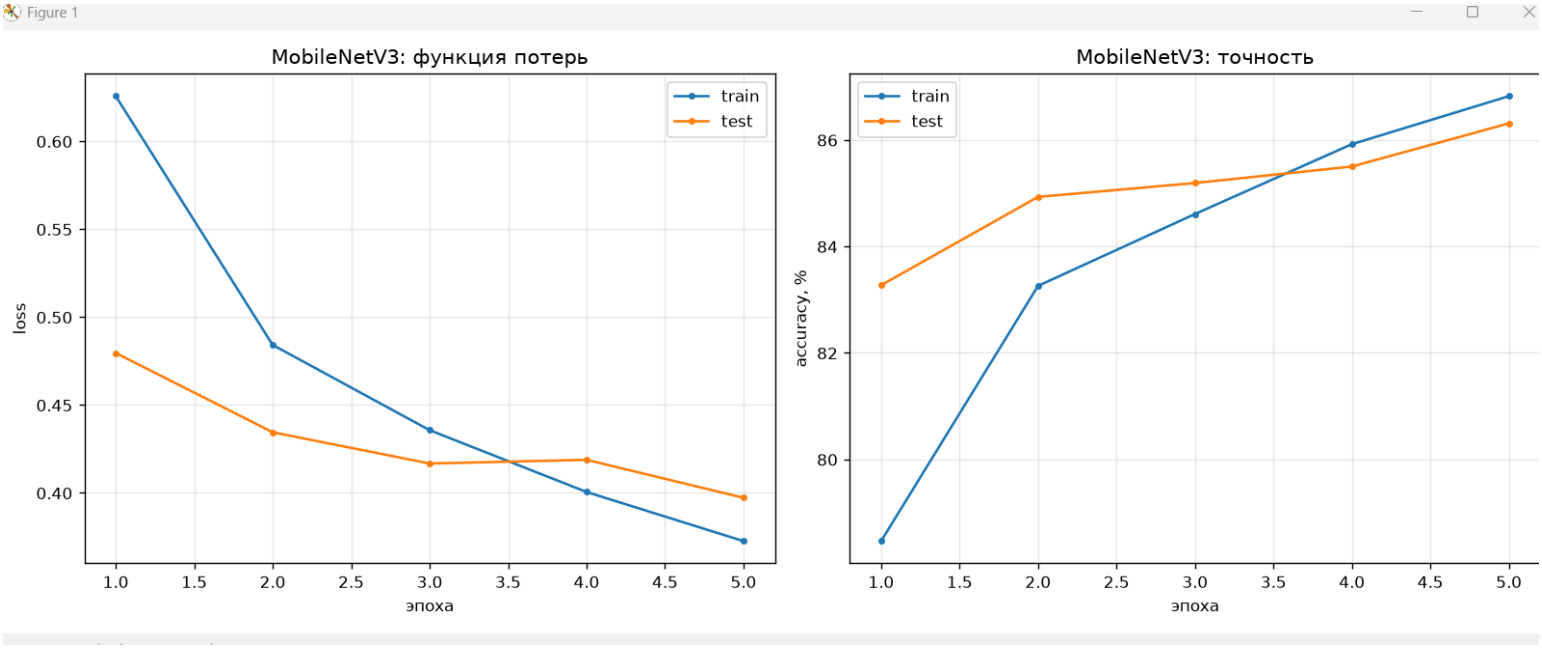
Отставание от ориентиров

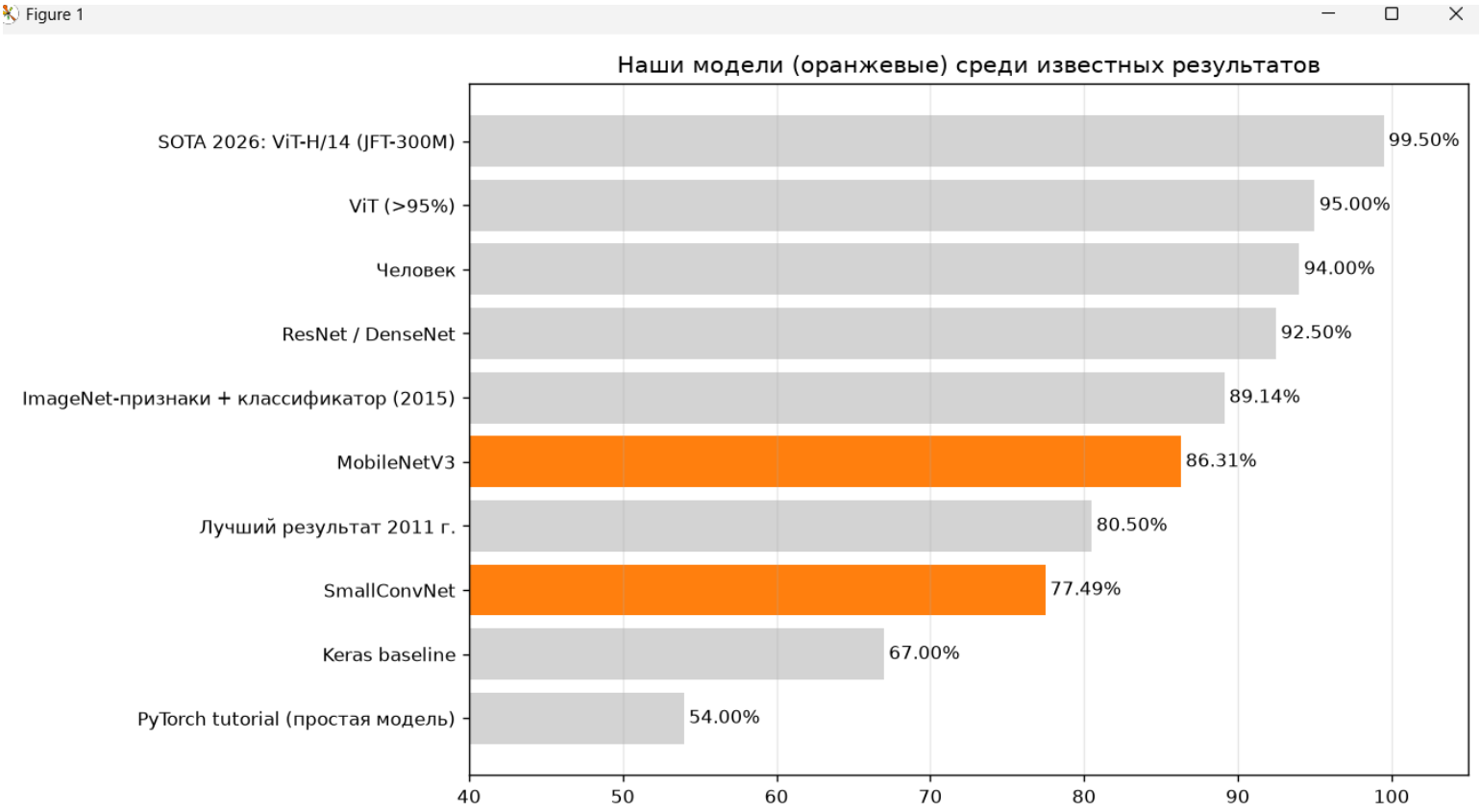
SmallConvNet: 77.49% | до человека: 16.51 п.п. | до SOTA: 22.01 п.п.

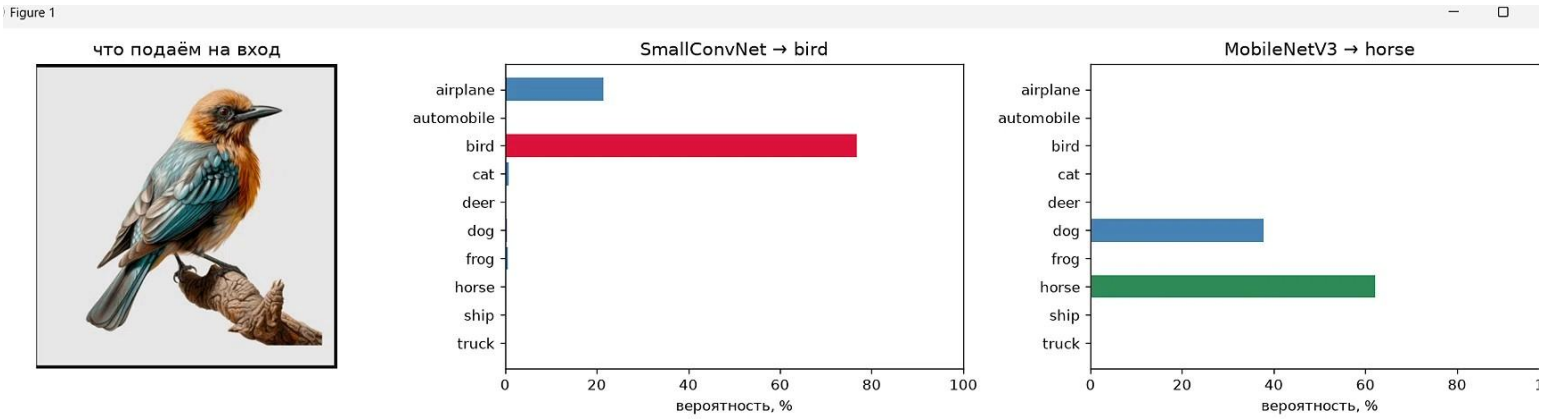
MobileNetV3 : 86.31% | до человека: 7.69 п.п. | до SOTA: 13.19 п.п.

Выигрыш MobileNetV3 над своей CNN: +8.82 п.п.

Process finished with exit code 0







Вывод: В ходе лабораторной сравнил самодельную сверточную сеть и предобученную MobileNetV3 на датасете CIFAR-10. Простая самодельная модель показала достойный результат в 77.49%, но использование готовых весов (Transfer Learning) дало колоссальный прирост: MobileNetV3 достигла точности 86.31% всего за 5 эпох, обучая лишь малую часть своих параметров, что наглядно доказывает, что брать готовую сеть — это гораздо быстрее и эффективнее, чем учить с нуля. Однако тест на реальной фотографии птицы выявил интересную проблему: самодельная сеть верно распознала птицу, а «умная» MobileNetV3 ошиблась, спутав её с лошадью, так как она привыкла к другим картинкам (высокого разрешения из ImageNet), а замороженные слои не дали ей адаптироваться к новым данным. В

итоге, чтобы в будущем выжать максимум, нужно не просто менять последний слой, а разморозить верхние слои предобученной сети, добавить больше разнообразия в картинки при обучении и плавно снижать скорость обучения (learning rate), чтобы модель не начинала переобучаться к концу тренировки.